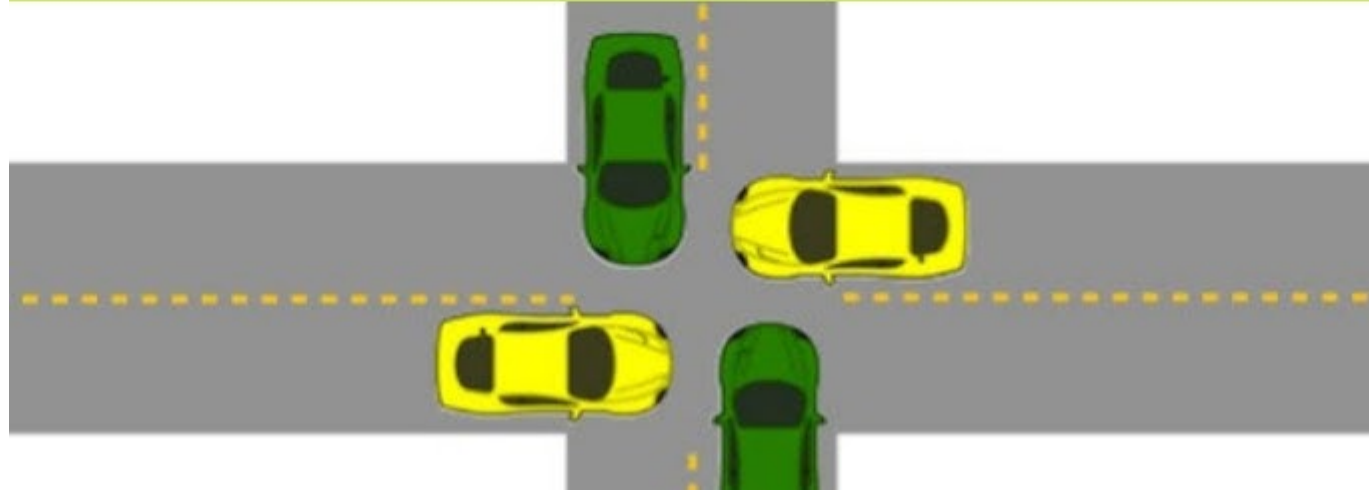




DEADLOCK



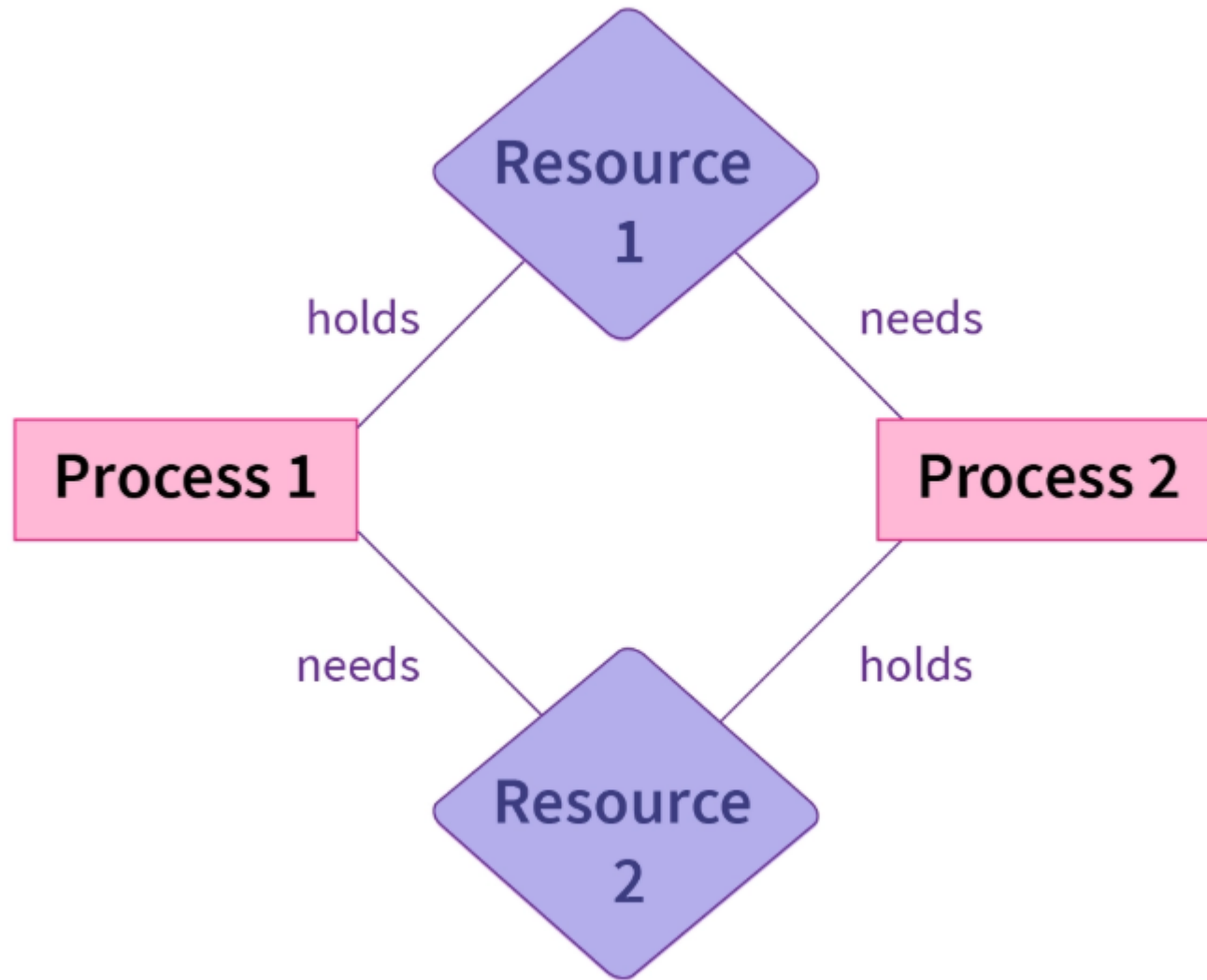
Concept of Deadlock



Deadlock

- Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.
- Or
- A deadlock is a situation in which more than one process is blocked because it is holding a resource and also requires some resource that is acquired by some other process. Therefore, none of the processes gets executed.





Deadlock

- System consists of a finite number of resources to be distributed among a number of competing processes.
- The resources are partitioned into several types, each of which consists of some number of identical instances.
- Memory space, CPU cycles, files, and I/O devices (such as printers) are examples of resource types.

Deadlock

- Under the normal mode of operation, a process may utilize a resource in only the following sequence:
- **Request:** If the request cannot be granted immediately (for example, the resource is being used by another process), then the requesting process must wait until it can acquire the resource.
- **Use:** The process can operate on the resource (for example, if the resource is a printer, the process can print on the printer).
- **Release:** The process releases the resource.

Deadlock Characterization

In a deadlock, processes never finish executing and system resources are tied up, preventing other jobs from starting.

Necessary Conditions

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- 1. Mutual exclusion:** At least one resource must be held in a non-shareable mode; that is, only one process at a time can use the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.
- 2. Hold and wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.

Deadlock Characterization

- 3. No pre-emption:** Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- 4. Circular wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , and finally P_n is waiting for a resource that is held by P_0 . This creates a circular relation between all these processes and they have to wait forever to be executed.

Resource-Allocation Graph

- Deadlocks can be described more precisely in terms of a directed graph called a system **resource-allocation graph**. This graph consists of a set of vertices V and a set of edges E
- The set of vertices V is partitioned into two different types of nodes
- $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system, and $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.

A directed edge $P_i \longrightarrow R_j$ is called a **request edge**; a directed edge $P_i \longleftarrow R_j$ is called an **assignment edge**.

Pictorially, we represent each process P_i as a circle, and each resource R_i as square

Resource-Allocation Graph

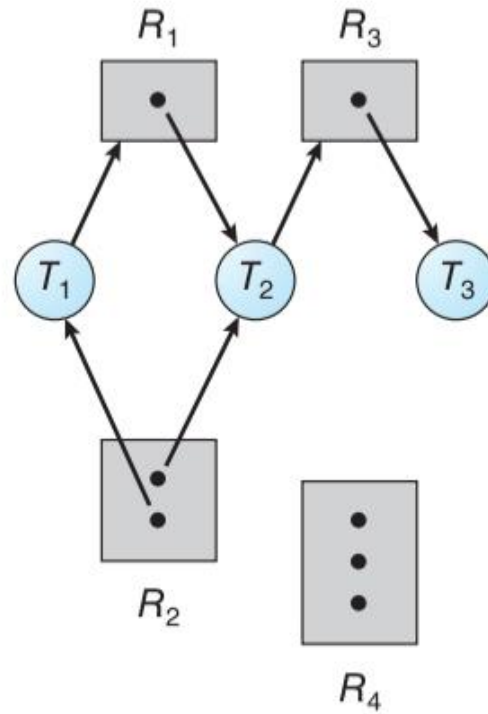


Figure 8.4 Resource-allocation graph.

Resource-Allocation Graph

The sets T, R, and E:

- $T = \{T1, T2, T3\}$

- $R = \{R1, R2, R3, R4\}$

$\{T1 \rightarrow R1, T2 \rightarrow R3, R1 \rightarrow T2, R2 \rightarrow T2, R2 \rightarrow T1, R3 \rightarrow T3\}$

- Resource instances:

- One instance of resource type R1

- Two instances of resource type R2

- One instance of resource type R3

- Three instances of resource type R4

- Process states:

- T1 is holding an instance of resource type R2 and is waiting for an instance of resource type R1.

- T2 is holding an instance of R1 and an instance of R2 and is waiting for an instance of R3.

- T3 is holding an instance of R3.

Resource-Allocation Graph

$+1 \leq +1 \leq +3 \leq +2 \leq +1$

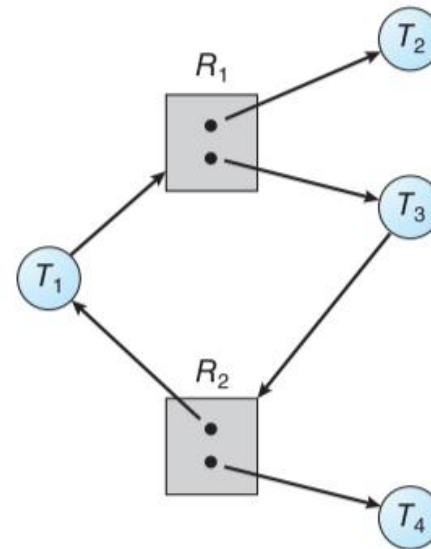


Figure 8.6 Resource-allocation graph with a cycle but no deadlock.

Resource-Allocation Graph

if a resource-allocation graph does not have a cycle, then the system is not in a deadlocked state. If there is a cycle, then the system may or may not be in a deadlocked state

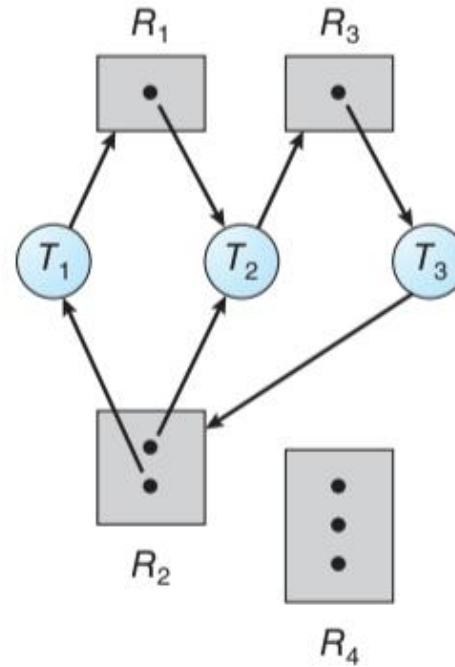
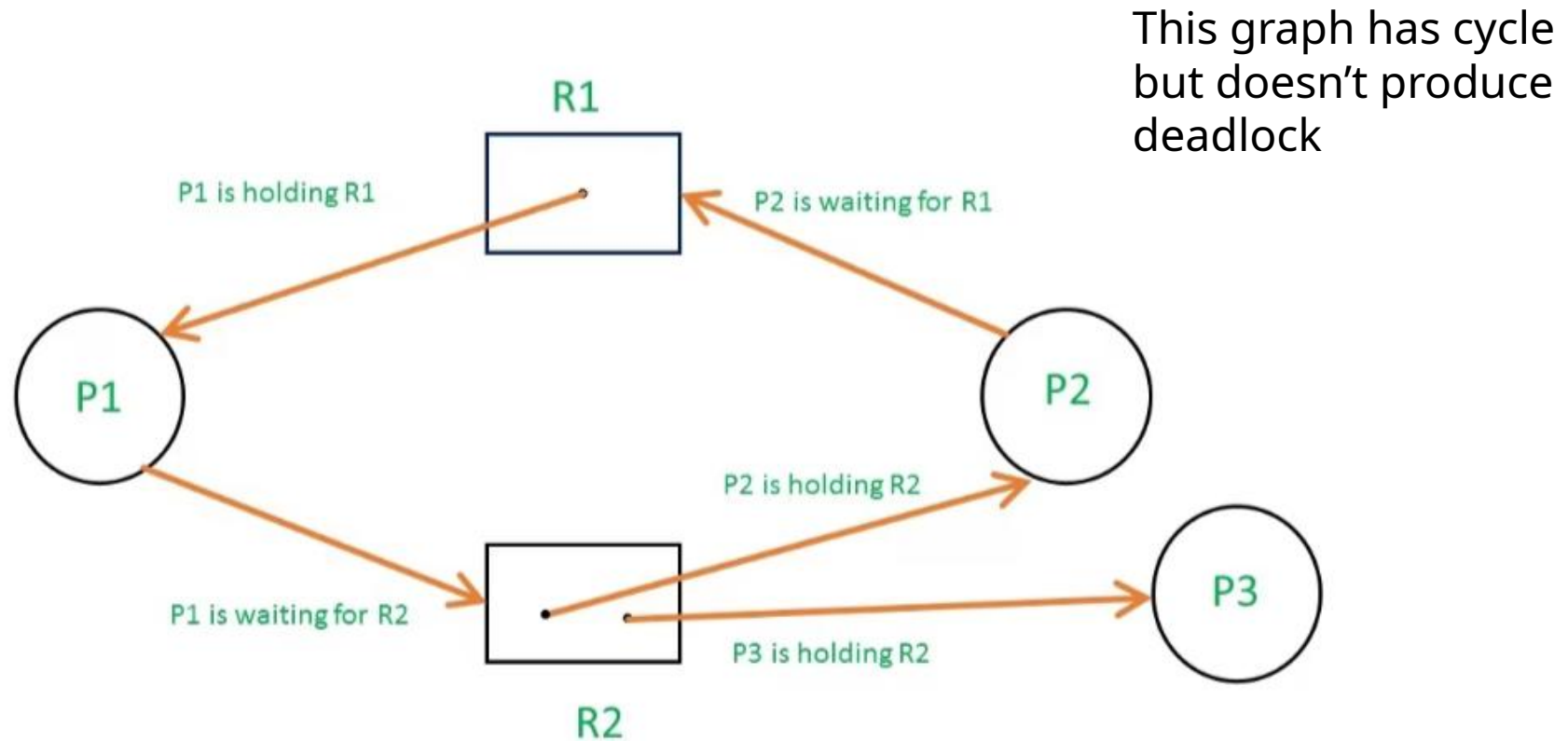


Figure 8.5 Resource-allocation graph with a deadlock.

Find Resource Allocation Graph

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	0	0

Does this graph produce deadlock?

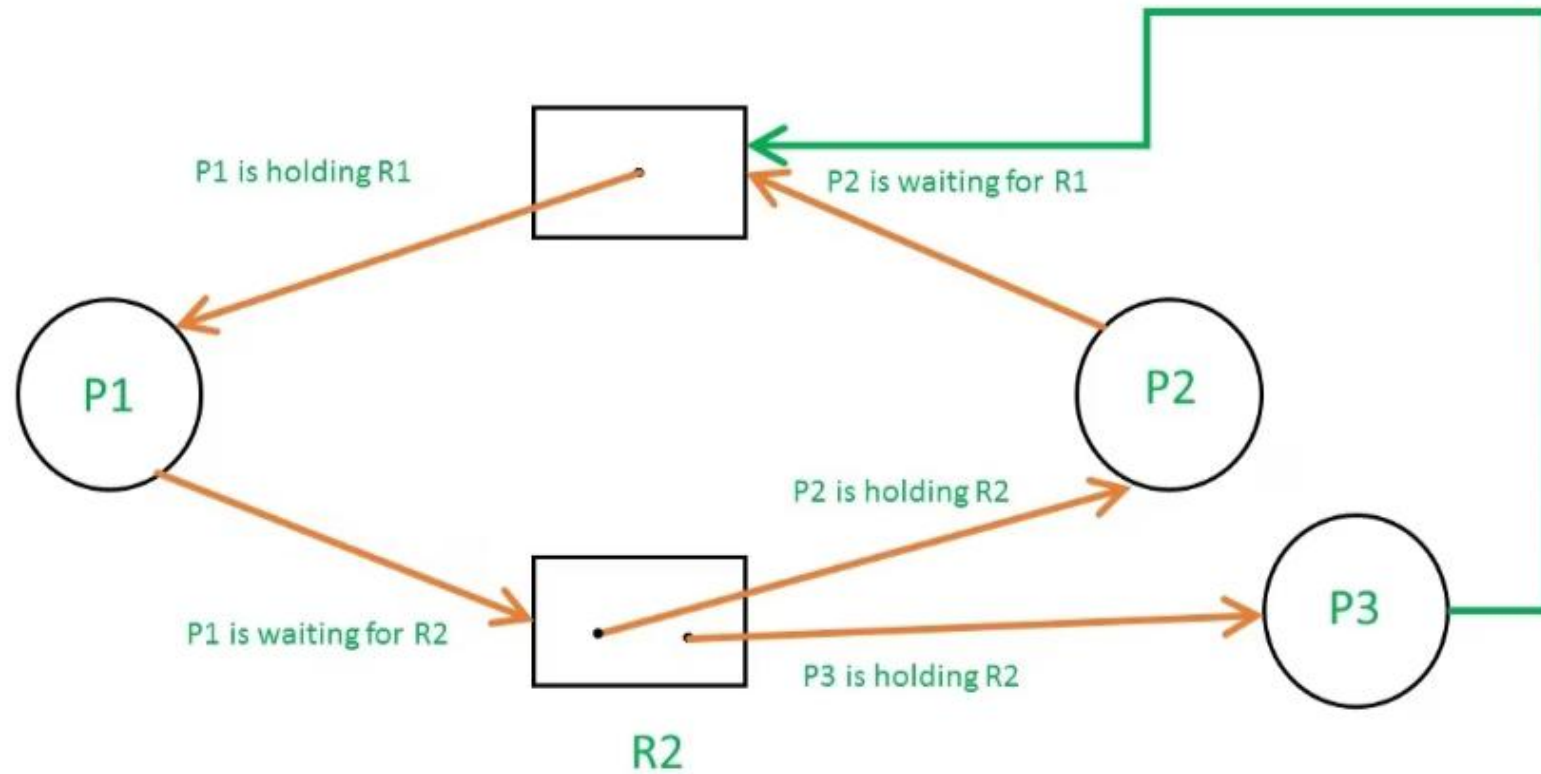


Find Resource Allocation Graph

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	1	0

Does this graph produce deadlock?

Deadlock



Methods for Handling Deadlocks

- To ensure that deadlocks never occur, the system can use either a deadlock prevention or a deadlock-avoidance scheme.
- **Deadlock prevention** provides a set of methods to ensure that at least one of the necessary conditions cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.
- **Deadlock avoidance** requires that the operating system be given additional information in advance concerning which resources a process will request and use during its lifetime.
- With this additional knowledge, the operating system can decide for each request whether or not the process should wait.
- To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

Deadlock Prevention

1. Eliminating Mutual Exclusion

How it works: Processes cannot have exclusive access to non-sharable resources.

Application: For some resources, like shared read-only files, this is possible. However, for non-sharable resources like printers, this condition is difficult to break.

2. Eliminating Hold and Wait

How it works: A process must request and be granted all its required resources upfront, or it must release all held resources before requesting new ones.

Application:

Request all at once: A process requests all its needed resources before it starts.

Release held resources: A process must release all its currently held resources before requesting new ones.

- **Eliminating hold and wait**

- ❖ Request All Resources at Once

- How it works: Before a process begins its execution, it requests all the resources it anticipates needing for its entire task.
- Benefit: If a process doesn't get all its required resources upfront, it waits without holding any resources, thereby eliminating the hold and wait condition.
- Drawback: This is often impractical in real computer systems because processes cannot predict all future resource needs. It can also lead to low device utilization because resources are allocated but may remain idle.

- ❖ Release Resources Before Requesting

- How it works: A process must release all resources it currently holds before it makes a request for any new resources.
- Benefit: This directly prevents a process from holding some resources while waiting for others, breaking the hold and wait cycle.
- Drawback: This approach can lead to starvation, where a process needing multiple resources might have to wait for a very long time because it keeps releasing and re-requesting resources, as at least one needed resource is always in use by another process.

Deadlock Prevention

3. Eliminating No Preemption

How it works: If a process holds resources and needs more that cannot be immediately allocated, the OS can forcibly take back (preempt) the resources it already holds.

Application: This allows the operating system to reallocate resources to other processes and avoid a process holding resources indefinitely.

4. Eliminating Circular Wait

How it works: A total ordering is imposed on all resources, and processes must request resources in a strictly increasing or decreasing order.

Application: Each resource is assigned a unique number (a priority). A process can only request a resource with a higher number than the resource it currently holds. If it needs a resource with low priority, it should release the currently high priority resource.

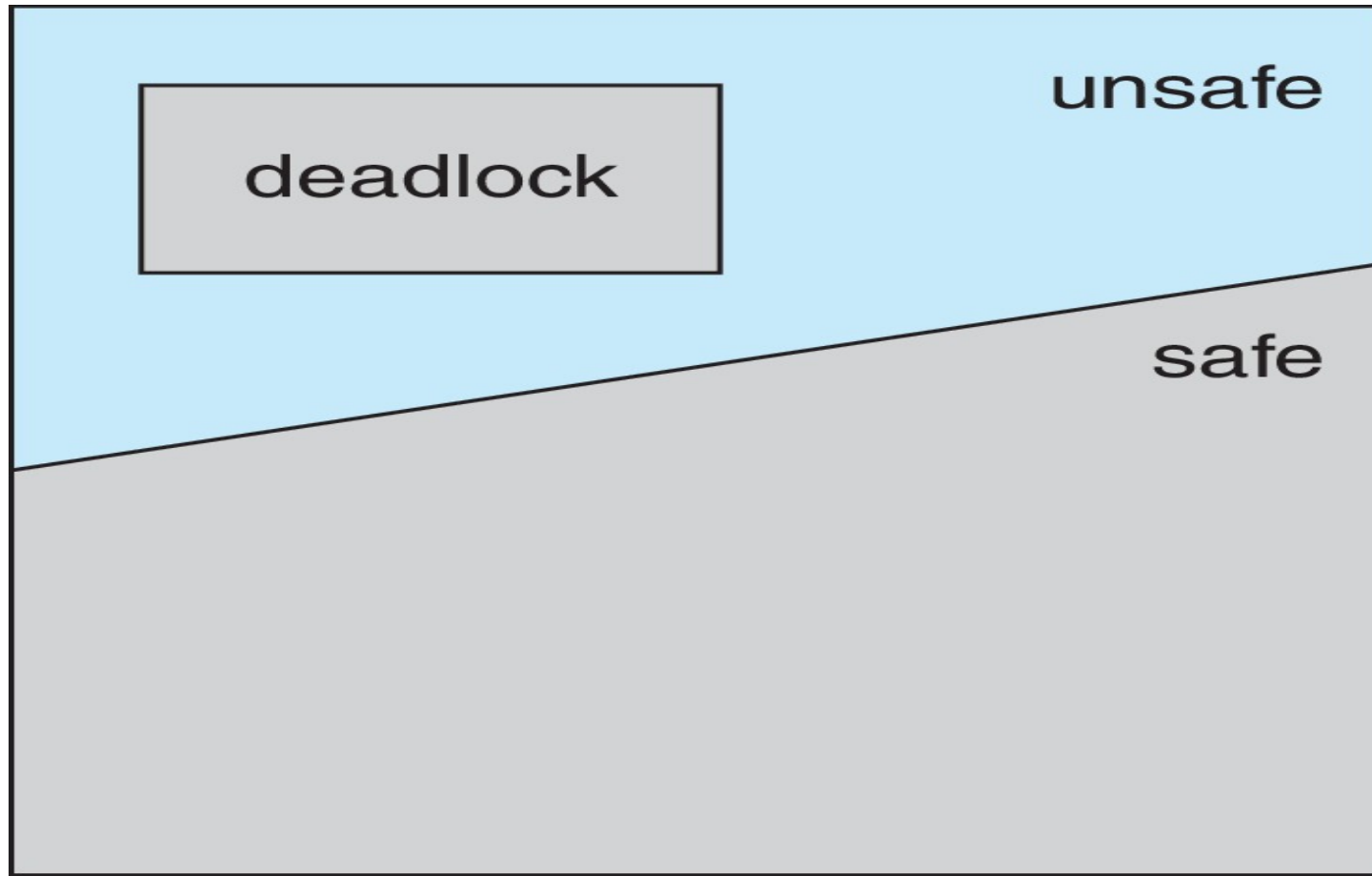
Deadlock Avoidance

- Requires that the system has some priori information available
- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

Basic facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, unsafe, deadlocked state



Basic facts

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the Banker's Algorithm

Deadlock Avoidance-Bankers Algorithm

- Banker's algorithm is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation for the predetermined maximum possible amounts of all resources.
- Helps the OS to successfully share the resources between all the processes. It is called the banker's algorithm because bankers need a similar algorithm to allocate loans to customers

Data structures for Bankers algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process T_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then T_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively.
Initialize:
 $Work = Available$
 $Finish[i] = false$ for $i = 0, 1, \dots, n-1$
2. Find an i such that both:
 - (a) $Finish[i] = false$
 - (b) $Need_i \leq Work$If no such i exists, go to step 4
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2
4. If $Finish[i] == true$ for all i , then the system is in a safe state

Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process T_i . If **$Request_i[j] = k$** then process T_i wants **k** instances of resource type **R_j**

1. If **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise T_i must wait, since resources are not available
3. Pretend to allocate requested resources to T_i by modifying the state as follows:

$Available = Available - Request_i$;

$Allocation_i = Allocation_i + Request_i$;

$Need_i = Need_i - Request_i$;

- If safe $\Rightarrow$ the resources are allocated to T_i
- If unsafe $\Rightarrow T_i$ must wait, and the old resource-allocation state is restored

Example of Bankers Algorithm

- 5 process T_0 through T_4 ;
3 resource types:
A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time t_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

What will be the content of the Need matrix?

The content of the matrix Need is defined to be Max – Allocation

- $\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Is the system in a safe state? If Yes, then what is the safe sequence?

$m=3, n=5$ Step 1 of Safety Algo

Work = Available

Work =

3	3	2
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

For $i = 0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and **Need₀ > Work**

So P₀ must wait

For $i = 1$ Step 2

Need₁ = 1, 2, 2

Finish [1] is false and **Need₁ < Work**

So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

A	B	C
5	3	2

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

For $i = 2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and **Need₂ > Work**

So P₂ must wait

For $i = 3$ Step 2

Need₃ = 0, 1, 1

Finish [3] = false and **Need₃ < Work**

So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

A	B	C
7	4	3

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

For $i = 4$ Step 2

Need₄ = 4, 3, 1

Finish [4] = false and **Need₄ < Work**

So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

A	B	C
7	4	5

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

For $i = 0$ Step 2

Need₀ = 7, 4, 3

Finish [0] is false and **Need₀ < Work**

So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

A	B	C
7	5	5

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

For $i = 2$ Step 2

Need₂ = 6, 0, 0

Finish [2] is false and **Need₂ < Work**

So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

A	B	C
10	5	7

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for $0 \leq i \leq n$

Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

What will happen if process P_1 requests one additional instance of resource type A and two instances of resource type C?

P_1 Request (1,0,2)

$\begin{matrix} & A & B & C \\ \text{Request}_1 = & 1, & 0, & 2 \end{matrix}$

To decide whether the request is granted we use Resource Request algorithm

$\begin{matrix} & A & B & C \\ 1, 0, 2 & 1, 2, 2 \\ \text{Request}_1 < \text{Need}_1 \end{matrix}$
✓ Step 1

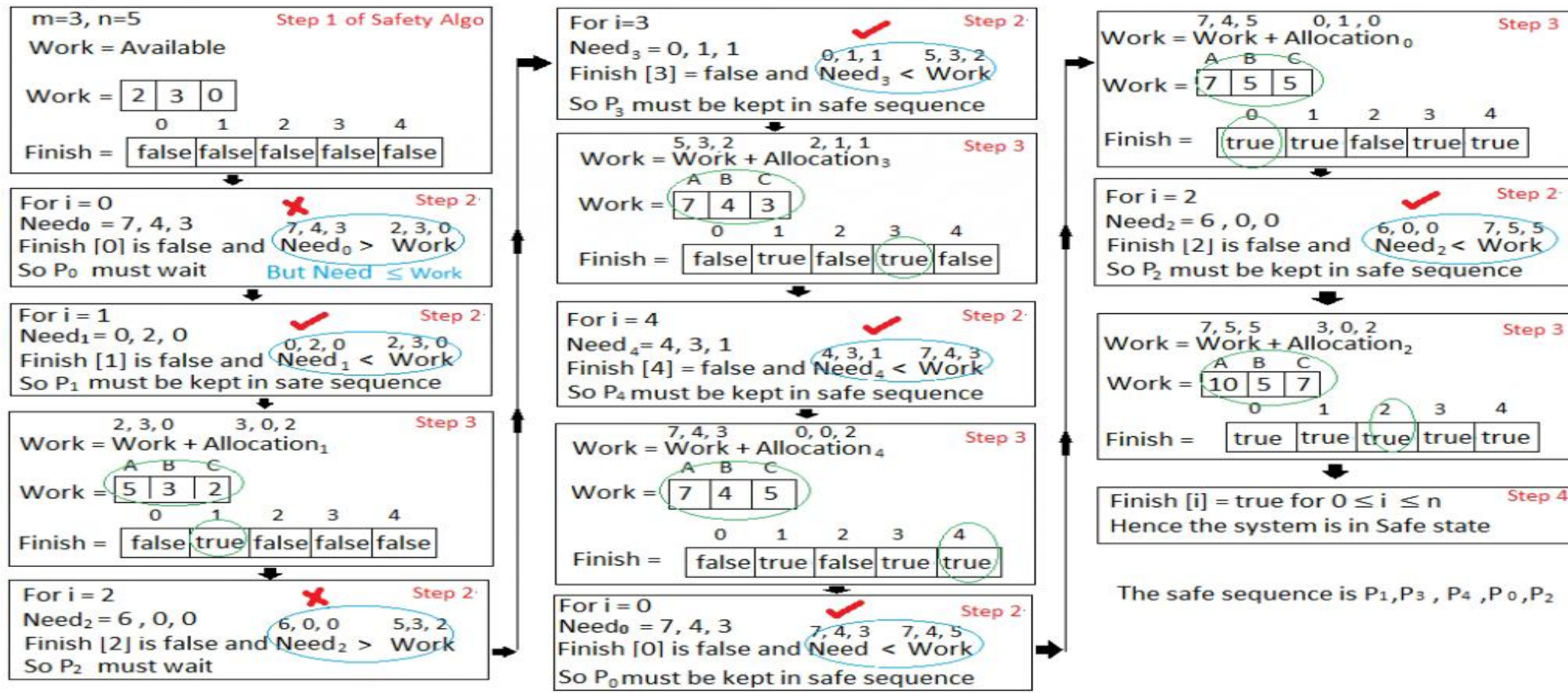
$\begin{matrix} & A & B & C \\ 1, 0, 2 & 3, 3, 2 \\ \text{Request}_1 < \text{Available} \end{matrix}$
✓ Step 2

Step 3

Available = Available – Request₁
 Allocation₁ = Allocation₁ + Request₁
 Need₁ = Need₁ - Request₁

Process	Allocation	Need	Available
	A B C	A B C	
P ₀	0 1 0	7 4 3	2 3 0
P ₁	3 0 2	0 2 0	
P ₂	3 0 2	6 0 0	
P ₃	2 1 1	0 1 1	
P ₄	0 0 2	4 3 1	

We must determine whether this new system state is safe. To do so, we again execute Safety algorithm



Deadlock Detection

1)Single Instance of Each Resource Type

- If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource allocation graph, called a wait-for graph.
- We obtain this graph from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges.

Deadlock Detection

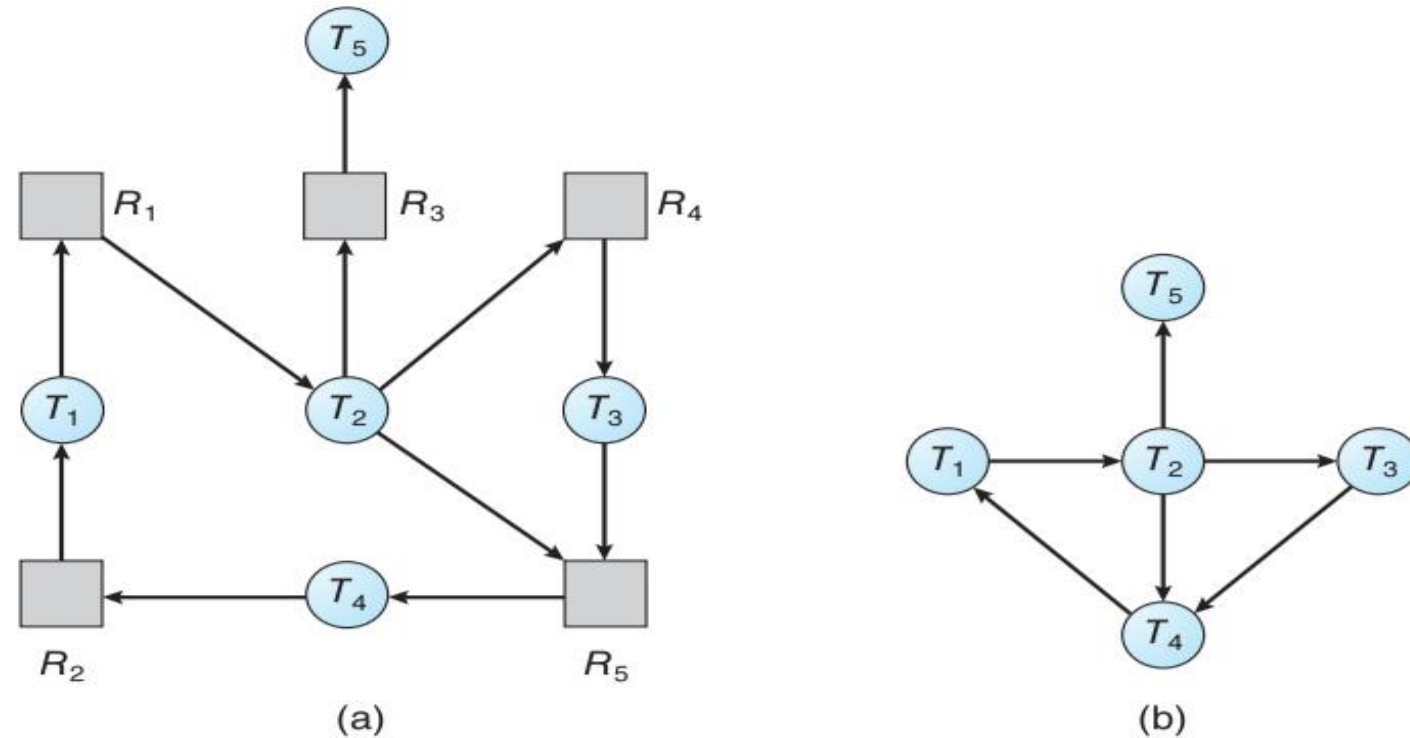


Figure 8.11 (a) Resource-allocation graph. (b) Corresponding wait-for graph.

Deadlock Detection

- an edge from T_i to T_j in a wait-for graph implies that process T_i is waiting for T_j to release a resource that T_i needs.
- An edge $T_i \rightarrow T_j$ exists in a wait-for graph if and only if the corresponding resource-allocation graph contains two edges $T_i \rightarrow R_q$ and $R_q \rightarrow T_j$ for some resource R_q .

Deadlock Detection

- A deadlock exists in the system if and only if the wait-for graph contains a cycle.
- To detect deadlocks, the system needs to maintain the wait-for graph and periodically invoke an algorithm that searches for a cycle in the graph.

Deadlock Detection

2) Several Instances of a Resource Type

Available. A vector of length m indicates the number of available resources of each type

Allocation. An $n \times m$ matrix defines the number of resources of each type currently allocated to each process

Request. An $n \times m$ matrix indicates the current request of each process. If $\text{Request}[i][j]$ equals k , then T_i is requesting k instances of resource type R_j

Deadlock Detection

1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize $Work = Available$. For $i = 0, 1, \dots, n-1$, if $Allocation_i \neq 0$, then $Finish[i] = false$. Otherwise, $Finish[i] = true$.
2. Find an index i such that both
 - a. $Finish[i] == false$
 - b. $Request_i \leq Work$If no such i exists, go to step 4.
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
Go to step 2.
4. If $Finish[i] == false$ for some i , $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $Finish[i] == false$, then thread T_i is deadlocked.

Deadlock Detection

- **Step 1 –**

- Let Work(vector) length = m
- Finish(vector) length = n
- Initialize Work= Available.
- For $i=0, 1, \dots, n-1$, if Allocation_i = 0, then Finish[i] = true; otherwise, Finish[i]= false.

- **Step 2 –**

- Find an index i such that both
 - Finish[i] == false
 - Request_i <= Work
- If no such i exists go to step 4.

- **Step 3 –**

- Work= Work + Allocation_i
- Finish[i]= true
- Go to Step 2.

- **Step 4 –**

- If Finish[i]== false for some i, $0 \leq i < n$, then the system is in a deadlocked state. Moreover, if Finish[i]==false the process P_i is deadlocked.
- Otherwise no deadlock.

Deadlock Detection

Example

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

A has 7 instances, B has 2 instances, C has 6 instances

Deadlock Recovery

- When a detection algorithm determines that a deadlock exists, several alternatives exist.

1) Process Termination

To eliminate deadlocks by aborting a process, we use one of two methods.

a) Abort all deadlocked processes: This method clearly will break the deadlock cycle, but at a great expense; these processes may have computed for a long time, and the results of these partial computations must be discarded and probably recomputed later

Deadlock Recovery

- **b) Abort one process at a time** until the deadlock cycle is eliminated: This method incurs considerable overhead, since, after each process is aborted, a deadlock-detection algorithm must be invoked to determine whether any processes are still deadlocked

Deadlock Recovery

2) Resource Pre-emption

- To eliminate deadlocks using resource pre-emption, we successively pre-empt some resources from processes and give these resources to other processes until the deadlock cycle is broken
- If pre-emption is required to deal with deadlocks, then three issues need to address
- 1. **Selecting a victim:** Which resources and which processes are to be preempted?
- As in process termination, we must determine the order of pre-emption to minimize cost. Cost factors may include such parameters as the number of resources a deadlock process is holding, and the amount of time a deadlocked process has thus far consumed during its execution.

Deadlock Recovery

2. Rollback: If we pre-empt a resource from a process, what should be done with that process?

- Clearly, it cannot continue with its normal execution; it is missing some needed resource. We must roll back the process to some safe state, and restart it from that state.

3. Starvation: How do we ensure that starvation will not occur?

In a system where victim selection is based primarily on cost factors, it may happen that the same process is always picked as a victim. As a result, this process never completes its designated task, a starvation situation that needs to be dealt with in any practical system. Clearly, we must ensure that a process can be picked as a victim only a (small) finite number of times.